



UNIVERSITÉ ABDELMALEK ESSAADI
École Nationale des Sciences Appliquées d'Al
Hoceima
Département Mathématique et Informatique
Filière : TDIA



Module : J2EE, Java EE (JEE) ou Jakarta EE

Compte Rendu : TP N° 6 – Projet Spring MVC, IOC, JSP & JSTL



Réalisé par : Taouni Ouïam

Supervisé par : Pr. Mohamed CHERRADI

Année universitaire 2024/2025

Table des matières

1	Objectif du laboratoire	2
2	Architecture du projet	3
3	Flux utilisateur et implémentation	4
3.1	Accès à l'application	4
3.2	Gestion des produits	5
4	Beans et configuration Spring (IoC)	6
4.1	Configuration XML des beans	6
4.2	Configuration Spring MVC (application-servlet-config.xml)	6
4.3	Configuration web.xml	7
4.4	Principe de l'IoC (Inversion of Control)	7
5	Conclusion	8

1 Objectif du laboratoire

Ce laboratoire avait pour objectif de développer une application web dynamique de type CRUD (Create, Read, Update, Delete) permettant la gestion des produits à l'aide du framework Spring MVC. L'application offre les principales fonctionnalités de gestion, notamment l'affichage de la liste des produits, l'ajout, la modification et la suppression d'un produit. Chaque produit est défini par un identifiant, un nom, une description et un prix.

Ce projet vise à mettre en pratique :

- Le framework Spring MVC
- L'Injection de dépendances (IoC – Inversion of Control)
- L'architecture en couches (3-tiers)
- L'utilisation de JSP + JSTL
- La configuration via XML et annotations

2 Architecture du projet

Le projet est basé sur une architecture 3-tiers (MVC) :

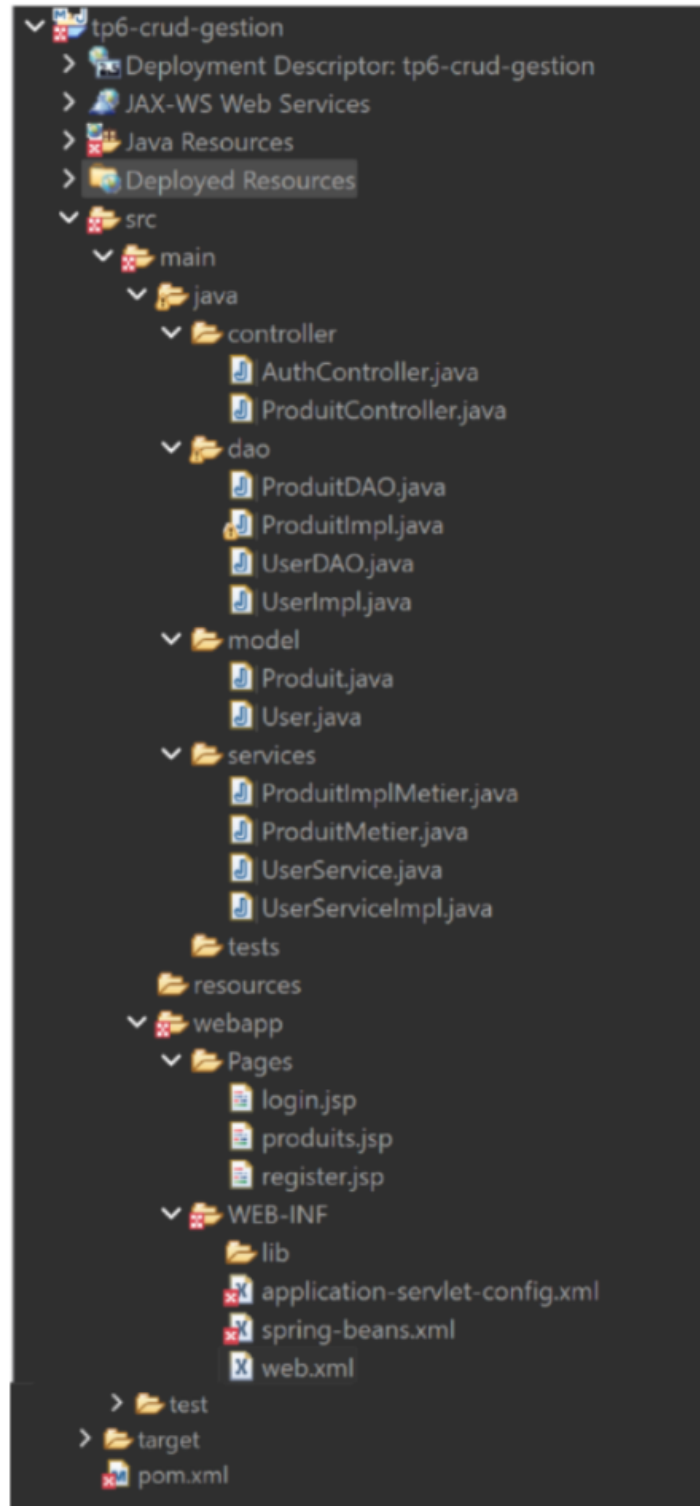


Figure 1 : Architecture 3-tiers du projet

3 Flux utilisateur et implémentation

3.1 Accès à l'application

L'utilisateur accède à l'application. L'accès à l'application est sécurisé par un système d'authentification permettant de contrôler les utilisateurs autorisés.

Ce module inclut les fonctionnalités de connexion (login) et d'inscription (register).

ou

Login

[Create account](#)

Register

[Already have an account? Login](#)

```
package controller;
import javax.servlet.http.HttpSession;

@Controller
public class AuthController {

    @Autowired
    UserService userService;

    @RequestMapping(value="/login", method=RequestMethod.GET)
    public String showLoginPage() {
        return "login";
    }

    @RequestMapping(value="/auth", method=RequestMethod.POST)
    public String login(@RequestParam String username,
        @RequestParam String password,
        HttpSession session,
        Model model) {
        User user = userService.login(username, password);
        if (user != null) {
            session.setAttribute("loggedUser", user);
            return "redirect:/index";
        }
        return "redirect:/login?error=true";
    }

    @RequestMapping(value="/register", method=RequestMethod.GET)
    public String showRegisterPage() {
        return "register";
    }

    @RequestMapping(value="/doRegister", method=RequestMethod.POST)
    public String register(@RequestParam String username,
        @RequestParam String password,
        @RequestParam String confirmPassword,
        Model model) {
        if (!password.equals(confirmPassword)) {
```

Figure 2 : Interface de connexion et d'inscription

3.2 Gestion des produits

La gestion des produits constitue le cœur fonctionnel de l'application et repose sur une architecture en couches utilisant Spring MVC. Ce module est implémenté à travers un contrôleur de produits (ProduitController) qui assure la communication entre l'interface utilisateur et la couche métier. Il permet de gérer les opérations CRUD (ajout, modification, suppression et affichage des produits) en s'appuyant sur les services associés.

```
@RequestMapping(value="/searchProduct")
public String searchProduct(Model model, @RequestParam(value = "idProduit") Long id)
    if (!isLoggedIn(session)) return "redirect:/login";

    List<Produit> liste = new ArrayList<Produit>();
    Produit p = services.getProduitById(id);
    if (p != null) {
        liste.add(p);
    }
    model.addAttribute("listeProduit", liste);
    model.addAttribute("idProduit", id);
    model.addAttribute("currentUser", session.getAttribute("loggedUser"));
    return "products";
}

@RequestMapping(value="/addProduct")
public String addProduct(Model model, Produit p, HttpSession session){
    if (!isAdmin(session)) return "redirect:/index?unauthorized=true";

    services.addProduit(p);
    model.addAttribute("listeProduit", services.getAllProduits());
    model.addAttribute("currentUser", session.getAttribute("loggedUser"));
    return "products";
}

@RequestMapping(value="/deleteProduit")
public String supprimerProduit(Model model, @RequestParam Long id, HttpSession ses
    if (!isAdmin(session)) return "redirect:/index?unauthorized=true";

    services.deleteProduit(id);
    model.addAttribute("listeProduit", services.getAllProduits());
    model.addAttribute("currentUser", session.getAttribute("loggedUser"));
    return "products";
}

@RequestMapping(value="/editProduit", method=RequestMethod.GET)
public String editProduit(Model model, @RequestParam Long id, HttpSession session)
```

Gestion des Produits admin (ADMIN) Logout

Search by ID Search

Nom Description Prix Add

ID	Nom	Description	Prix	Actions
1	PC	Dell XPS 13	15000.0	Edit Delete
2	Water	Sidi ali	6.0	Edit Delete
3	Smart Watch	Samsung	8000.0	Edit Delete

Figure 4 : Interface de gestion des produits

4 Beans et configuration Spring (IoC)

Le projet utilise Spring IoC (Inversion of Control) pour gérer les dépendances.

4.1 Configuration XML des beans

On distingue deux types principaux de composants :

```
<bean id="produitDAO" class="dao.ProduitImpl" init-method="init"/>
<bean id="userDAO" class="dao.UserImpl" init-method="init"/>
```

Ils sont déclarés comme des beans Spring afin que leur cycle de vie soit entièrement géré par le conteneur.

```
<bean id="produitService" class="services.ProduitImplMetier">
    <property name="dao" ref="produitDAO"/>
</bean>
<bean id="userService" class="services.UserServiceImpl">
    <property name="userDao" ref="userDAO"/>
</bean>
```

L'injection de dépendances est réalisée dans ce fichier XML. Spring instancie les beans DAO puis les injecte automatiquement dans les services grâce aux balises <property>. Cela permet de relier les différentes couches sans instantiation manuelle.

4.2 Configuration Spring MVC (application-servlet-config.xml)

Cette configuration active le fonctionnement du framework Spring MVC. Le mécanisme de component-scan permet à Spring de détecter automatiquement les classes annotées dans les packages spécifiés.

```
<context:component-scan base-package="controller,services,dao,model"/>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/Pages"/>
    <property name="suffix" value=".jsp"/>
</bean>
```

4.3 Configuration web.xml

Le DispatcherServlet est le cœur de Spring MVC. Il intercepte toutes les requêtes HTTP et les redirige vers les contrôleurs appropriés. Il est également associé au fichier de configuration Spring MVC.

```
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/application-servlet-config.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
```

Figure 6 : Configuration web.xml

4.4 Principe de l'IoC (Inversion of Control)

Spring repose sur le principe d'Inversion of Control, où la création des objets n'est plus effectuée manuellement par le développeur, mais déléguée au conteneur Spring.

Les dépendances sont injectées automatiquement par Spring à l'aide de :

- L'injection par constructeur
- L'injection par setter
- L'injection par annotation (Autowired)

```
@Controller
public class ProduitController {
    @Autowired
    ProduitMetier services;
    // ...
}
```

Figure 7 : Schéma du principe IoC

5 Conclusion

Ce laboratoire a permis de concevoir et de développer une application web complète de type CRUD dédiée à la gestion des produits en utilisant le framework Spring MVC. À travers ce projet, nous avons mis en œuvre les principales opérations de gestion de données ainsi que les différents composants d'une architecture web structurée selon le modèle 3-tiers.

L'application illustre également l'utilisation des technologies JSP et JSTL pour l'affichage des données côté client. Ce travail a surtout été une opportunité de comprendre et d'appliquer les concepts fondamentaux de Spring, notamment le principe d'Inversion of Control (IoC), la gestion des dépendances, ainsi que la configuration des beans via XML et annotations.

Enfin, ce projet constitue une base solide pour le développement d'applications web plus avancées en Java, en renforçant les compétences en architecture logicielle et en framework Spring.